

Initializing an SD Card Part 3

In the previous tutorial, we learned how to use CMD0, CMD8 and CMD58. We have almost finished the initialization sequence. Looking at the diagram below, we see the next step is issuing ACMD41.

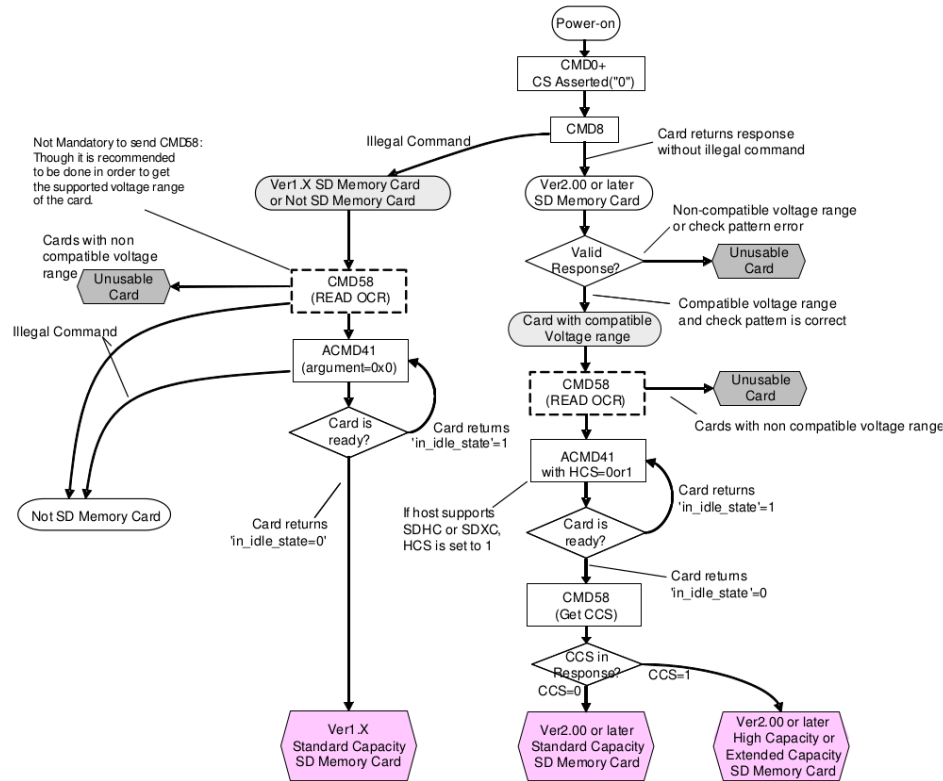


Figure 7-2 : SPI Mode Initialization Flow

ACMD41

ACMD41 - SD_SEND_OP_COND (send operating condition), is what starts the card's initialization process. However, there is a sneaky nuance here not shown in the diagram. You will notice that ACMD41 differs from the other commands we have seen in that its name begins with an A. This signifies that it is an application specific command. We let the SD card know that the command we are sending is application specific by first sending CMD55 - APP_CMD.

Here is the format for CMD55

CMD INDEX	SPI Mode	Argument	Resp	Abbreviation	Command Description
CMD55	Yes	[31:0] stuff bits	R1	APP_CMD	Defines to the card that the next command is an application specific command rather than a standard command

And here is the format for ACMD41

CMD INDEX	SPI Mode	Argument	Resp	Abbreviation	Command Description
ACMD41	Yes	[31]Reserved bit [30]HCS [29:0]Reserved bits	R1	SD_SEND_OP_COND	Sends host capacity support information and activates the card's initialization process. Reserved bits shall be set to '0'

We will use the following define statements to help us when sending these commands. Note that most bits in ACMD41 are reserved (which means set them to 1), but we have an option to set bit 30 to 1 to indicate we support high capacity cards. The CRC is ignored for these commands so we can set it to anything.

```
#define CMD55      55
#define CMD55_ARG  0x00000000
#define CMD55_CRC  0x00

#define ACMD41     41
#define ACMD41_ARG 0x40000000
#define ACMD41_CRC 0x00
```

You may note that ACMD41 uses command index 41 just like CMD41 would. In fact, there is no difference in the command itself that differentiates ACMD41 from CMD41. If CMD55 is not sent first, then sending a command with 41 in the first byte will be interpreted as CMD41, which is a reserved command and will result in an error response.

Both CMD55 and ACMD41 return R1, which we have already written code for. Thus, implementing these commands is extremely easy:

```
uint8_t SD_sendApp()
{
    // assert chip select
    SPI_transfer(0xFF);
    CS_ENABLE();
    SPI_transfer(0xFF);

    // send CMD0
    SD_command(CMD55, CMD55_ARG, CMD55_CRC);

    // read response
    uint8_t res1 = SD_readRes1();

    // deassert chip select
    SPI_transfer(0xFF);
    CS_DISABLE();
    SPI_transfer(0xFF);

    return res1;
}

uint8_t SD_sendOpCond()
{
    // assert chip select
    SPI_transfer(0xFF);
    CS_ENABLE();
    SPI_transfer(0xFF);

    // send CMD0
    SD_command(ACMD41, ACMD41_ARG, ACMD41_CRC);

    // read response
    uint8_t res1 = SD_readRes1();

    // deassert chip select
    SPI_transfer(0xFF);
    CS_DISABLE();
    SPI_transfer(0xFF);

    return res1;
}
```

As said before, ACMD41 starts the initialization process. In the startup sequence, we will continue to send ACMD41 (always preceded by CMD55) until the card responds with 'in_idle_state', which is R1 = 0x00.

When we finally receive R1 = 0x00, we need to send CMD58 to see the value of CCS, which will tell us if the card is a high capacity SD card (SDHC) or extended capacity SD card (SCXC).

Here is a modification to the main function from the previous tutorial with CMD55 and ACMD41 added. Download source code [here](#).

```
int main(void)
{
    // array to hold responses
    uint8_t res[5];

    // initialize UART
    UART_init(57600);
```

```

// initialize SPI
SPI_init();

// start power up sequence
SD_powerUpSeq();

// received char from UART
char c;

while(1)
{
    // print menu
    UART_puts("MENU\r\n");
    UART_puts("-----\r\n");
    UART_puts("0 - Send CMD0\r\n1 - Send CMD8\r\n2 - Send CMD58\r\n");
    UART_puts("3 - Send CMD55\r\n4 - Send ACMD41\r\n");
    UART_puts("-----\r\n");

    // get character from user
    c = UART_getc();

    if(c == '0')
    {
        // command card to idle
        UART_puts("Sending CMD0...\r\n");
        res[0] = SD_goIdleState();
        UART_puts("Response:\r\n");
        SD_printR1(res[0]);
    }
    else if(c == '1')
    {
        // send if conditions
        UART_puts("Sending CMD8...\r\n");
        SD_sendIfCond(res);
        UART_puts("Response:\r\n");
        SD_printR7(res);
    }
    else if(c == '2')
    {
        // send if conditions
        UART_puts("Sending CMD58...\r\n");
        SD_readOCR(res);
        UART_puts("Response:\r\n");
        SD_printR3(res);
    }
    else if(c == '3')
    {
        // command card to idle
        UART_puts("Sending CMD55...\r\n");
        res[0] = SD_sendApp();
        UART_puts("Response:\r\n");
        SD_printR1(res[0]);
    }
    else if(c == '4')
    {
        // command card to idle
        UART_puts("Sending ACMD41...\r\n");
        res[0] = SD_sendOpCond();
        UART_puts("Response:\r\n");
        SD_printR1(res[0]);
    }
    else
    {
        UART_puts("Unrecognized command\r\n");
    }
}
}

```

Now we run through the sequence as before, we send CMD0, followed by CMD8 and CMD58.

```
File Edit View Search Terminal Help
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
Sending CMD0...
Response:
      In Idle State
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
Sending CMD8...
Response:
      In Idle State
      Command Version: 00
      Voltage Accepted: 2.7-3.6V
      Echo: AA
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
Sending CMD58...
Response:
      In Idle State
      Card Power Up Status: BUSY
      VDD Window: 2.7-2.8, 2.8-2.9, 2.9-3.0, 3.0-3.1, 3.1-3.2, 3.2-3.3, 3.3-3.4, 3.4-3.5, 3.5-3.6
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
```

Now for the new part. We will send CMD55 followed by ACMD41 until the card tells us it is ready. Then we will send CMD58 one last time to get CCS.

```

File Edit View Search Terminal Help
Sending CMD55...
Response:
    In Idle State
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
Sending ACMD41...
Response:
    In Idle State
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
Sending CMD55...
Response:
    In Idle State
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
Sending ACMD41...
Response:
    Card Ready
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----
Sending CMD58...
Response:
    Card Ready
    Card Power Up Status: READY
    CCS Status: 1
    VDD Window: 2.7-2.8, 2.8-2.9, 2.9-3.0, 3.0-3.1, 3.1-3.2, 3.2-3.3, 3.3-3.4, 3.4-3.5, 3.5-3.6
-----
MENU
0 - Send CMD0
1 - Send CMD8
2 - Send CMD58
3 - Send CMD55
4 - Send ACMD41
-----

```

See that the first time we send ACMD41 the card reported in idle, but the second time it was ready. Also, now when we read OCR in CMD58, the card reports it has finished its power up process and the CCS bit is valid - in this case it is set to 1, meaning we are using a SDXC or SDHC card.

Initialization Function

Now that we can initialize a card interactively, we should place all of the commands in a function that will go through the process automatically. This requires we take a look at the power up diagram again:

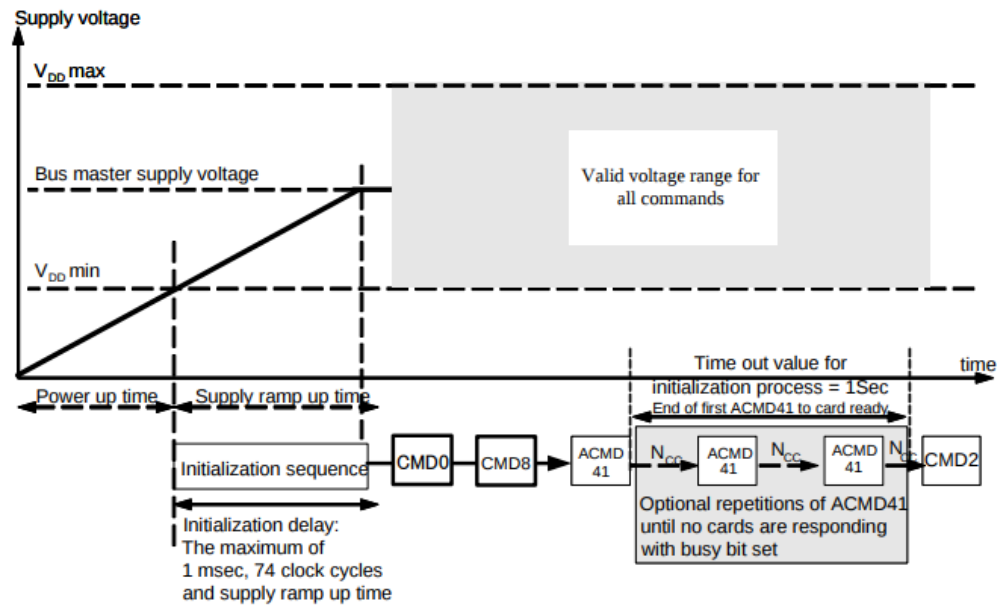


Figure 6-1: Power-up Diagram

Note now that the timeout value for initialization beginning from the end of the first ACMD41 is 1 sec. Doing this interactively you probably never ran into a problem, as the time between commands was only as fast as you typed them in. However, programmatically, we need to continue trying to initialize the card for at least a second. Here, I placed a delay of 10ms between each attempt, with a maximum number of 100 attempts. Of course this could be accomplished with a timer but for a simple initialization procedure this works well.

```
#define SD_SUCCESS 0
#define SD_ERROR 1

uint8_t SD_init()
{
    uint8_t res[5], cmdAttempts = 0;

    SD_powerUpSeq();

    // command card to idle
    while((res[0] = SD_goIdleState()) != 0x01)
    {
        cmdAttempts++;
        if(cmdAttempts > 10) return SD_ERROR;
    }

    // send interface conditions
    SD_sendIfCond(res);
    if(res[0] != 0x01)
    {
        return SD_ERROR;
    }

    // check echo pattern
    if(res[4] != 0xAA)
    {
        return SD_ERROR;
    }

    // attempt to initialize card
    cmdAttempts = 0;
    do
    {
        if(cmdAttempts > 100) return SD_ERROR;

        // send app cmd
        res[0] = SD_sendApp();

        // if no error in response
        if(res[0] < 2)
        {
            res[0] = SD_sendOpCond();
        }

        // wait
        _delay_ms(10);
    }
}
```

```
        cmdAttempts++;  
    }  
    while(res[0] != SD_READY);  
  
    // read OCR  
    SD_readOCR(res);  
  
    // check card is ready  
    if(!(res[1] & 0x80)) return SD_ERROR;  
  
    return SD_SUCCESS;  
}
```

And there you have it, initializing an SD Card in SPI mode. Click [here](#) for part 4 of the tutorial where we will go over read and write operations.

rjhcoding.com 2018